

后端第二题

Task 1

- Java 的八种基本数据类型分别是：**byte,short,int,long,float,double,char,boolean**
- 1. short 2字节 -32768 ~ 32767
 - 2. byte 1字节 -128-127
 - 3. int 4字节 约 -21 亿 ~ 21 亿 ($-2^{31} \sim 2^{31} - 1$)
 - 4. long 8字节 约 $-9.2 \times 10^{18} \sim 9.2 \times 10^{18}$
 - 5. float 4字节
 - 6. double 8字节
 - 7. char 2字节
 - 8. boolean 未精确定义

	基本类型	引用类型
存的是什么	直接存"值"	存"地址" (指向对象)
存在哪里	栈 (stack)	对象在堆 (heap) , 变量存引用
例子	<code>int a = 5</code> , a 里就是 5	<code>String s = "hi"</code> , s 里是"hi"的内存地址
比较	<code>==</code> 比较的是值	<code>==</code> 比较的是地址, 用 <code>.equals()</code> 比较内容

- byte、short、char 参与运算时，都是会先自动提升为 int类型，这是隐式类型转换的，不需要写 (int)c
- b的值为**52** 因为c 是 char 类型的字符 '0'，它实际存的是它的 Unicode/ASCII 码值为48 当 char 和 int 相加时，char 会隐式转换为 int，所以 a + c 实际上是 4 + 48.结果为 b = 52
- 结果为13 6 8 ++a为先加后用1 b++为先用后加1 c=6 + 7 = 13 a = 6（前面已经加1过），b = 8（后面 b++ 已经运行过），所以打印 6 8
- int 是 32 位，最高位是符号位。最大正值是 $2^{31} - 1 = 2147483647$ 二进制的 2147483647 是：01111111111111111111111111111111 如果给这个数再加 1，结果变成：10000000000000000000000000000000 此时最高位从 0 变成了 1，而最高位为 1 表示负数，所以这个结果在 Java 里也就是 -2147483648 这就是溢出了，结果超出了类型能容纳的范围，数字"绕了一圈"从最大值回到了最小值，符号位被进位占据，让正数相加看起来变成

了负数 举个例子: `int x = 2147483647; // 最大值 System.out.println(x + 1); // 输出 -2147483648`

- `float` 和 `double` 是通过 数值 = 符号位 × 尾数 × $2^{\text{指数}}$ 的方式存储
`float` 只有 32 位, `double` 64 位, 而分配给"尾数"的位数有限, 无法精确表示所有小数, 很多十进制小数 (比如 0.1、0.2) 在二进制里是无限循环小数 (类似十进制的 $1/3 = 0.33333...$), 只能保存一部分, 所以会产生微小误差
举例 `System.out.println(0.1 + 0.2);` // 输出的是 0.30000000000000004, 而不是 0.3
- `int` 是基本类型, `Integer` 是它的包装类 (引用类型)。Java 会自动在两者间转换
 1. `new` 每次都会在堆里新创建一个对象, `x` 和 `y` 指向两个不同的对象 (地址不同), `==` 比较引用类型的地址, 所以不相等
 2. `Integer.valueOf()` 会优先从缓存池里取。Java 对 -128 ~ 127 范围内的整数做了缓存, 18 在这个范围内, 所以 `z` 和 `k` 拿到的是缓存里的同一个对象, 地址相同, `==` 为 `true`。
 3. 300 超出了 -128 ~ 127 的缓存范围, `valueOf` 这时会 `new` 一个新的对象, `m` 和 `p` 是不同对象, 所以 `==` 为 `false`

Task 2

```
boolean isLeapYear(int year) {  
    if (year % 400 == 0) {           // 能被400整除, 一定是闰年  
        return true;  
    } else if (year % 100 == 0) {    // 能被100整除但不能被400整除, 不是闰年  
        return false;  
    } else if (year % 4 == 0) {      // 能被4整除但不能被100整除, 是闰年  
        return true;  
    } else {  
        return false;               // 其他都不是闰年  
    }  
}
```

- `if-else` 适合: 条件范围或复杂条件, 比如数值区间 (`x > 10 && x < 100`)、多种逻辑组合、条件判断是布尔表达式的场景。灵活性高, 几乎什么都能用 `switch-case` 适合: 对单个变量的"等值匹配", 例如根据数字、枚举、字符判断不同分支。当变量等于某个确切的值时, 用 `switch` 更清晰
- 不完全同意 从功能上讲, 任何 `switch` 都能改写成等价的 `if-else`, 两者在语义上都能实现"多分支选择" 从编译后的实现看, 很多 `switch` 最终也会被编译成类似跳转表或连续的比较判断,

与 if-else 在某些情况下类似 switch 有 break、贯穿 (fall-through) 语义，以及新版的模式匹配等独特语法，不是简单地把 if-else 换个写法

- 1.

```
import java.util.Scanner;

public class Fibonacci {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // 创建 Scanner，准备读键盘输入

        System.out.print("请输入要打印的斐波那契数个数：");
        int n = sc.nextInt(); // 读取用户输入的一个整数，存到 n

        // 打印前 n 个斐波那契数
        for (int i = 1; i <= n; i++) {
            System.out.print(fibonacci(i) + " ");
        }
        System.out.println(); // 最后换行，输出好看一点

        sc.close(); // 用完关闭 Scanner（好习惯，避免资源泄漏）
    }

    // 递归实现
    static int fibonacci(int n) {
        if (n == 1 || n == 2) {
            return 1;
        }
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

- 2.

```
import java.util.Scanner;

public class Fibonacci {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

        System.out.print("请输入要打印的斐波那契数个数：");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print(fibonacci(i) + " ");
        }
        System.out.println();

        sc.close();
    }

```

// 迭代实现

```

static int fibonacci(int n) {
    if (n == 1 || n == 2) {
        return 1;
    }
    int prev = 1;
    int curr = 1;
    for (int i = 3; i <= n; i++) {
        int next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
}

```

- 递归：函数调用自身函数来解决这个问题，把大问题拆成小问题，靠栈保存中间状态 迭代：用循环来不断重复执行一段代码，靠变量保存中间状态
- 1. 效率高：递归每次调用都要压栈、保存现场，有函数调用开销；迭代没有这些开销
- 2. 以斐波那契为例：递归会重复计算大量子问题（比如算 $F(5)$ 要算两次 $F(3)$ 、三次 $F(2)$ ），时间复杂度高达 $O(2^n)$ ；迭代只需 $O(n)$ ，快得多
-

在理论上，是的——所有循环都能改写成等价的递归（反之亦然，因为它们都能表达"重复执行" 但代价是：

1. 性能下降：递归有函数调用开销，通常比循环慢

2. 内存风险：深递归会占用大量栈空间，可能导致栈溢出
 3. 阅读难度下降：有些用循环，硬改成递归反而难懂
- // from: 源柱子, to: 目标柱子, aux: 辅助柱子

```
void hanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        // 只剩一个盘子，直接从 from 移到 to
        System.out.println(from + "->" + to);
        return;
    }
    // 1. 先把上面 n-1 个盘子从 from 移到 aux (借助 to)
    hanoi(n - 1, from, aux, to);
    // 2. 把最下面一个盘子从 from 移到 to
    System.out.println(from + "->" + to);
    // 3. 把 n-1 个盘子从 aux 移到 to (借助 from)
    hanoi(n - 1, aux, to, from);
}

// 题目要求的入口，只需要传 n
void hanoi(int n) {
    hanoi(n, 'A', 'C', 'B'); // 源A, 目标C, 辅助B
}
```

Task 3

•	Error	Exception
是谁的错	通常是 JVM、系统级的严重问题	通常是程序逻辑、外部条件导致的
例子	<code>OutOfMemoryError</code> （内存耗尽）、 <code>StackOverflowError</code>	<code>IOException</code> 、 <code>NumberFormatException</code> 、 <code>NullPointerException</code>
能否处理	通常无法也不应处理	应该捕获并处理
态度	让它崩，因为程序无法正常继续	积极 catch、修复、给出提示

	checked exception	unchecked exception
定义	编译期强制检查的异常	编译期不强制检查
继承自	<code>Exception</code> (但不包括 <code>RuntimeException</code>)	<code>RuntimeException</code> 及其子类
是否必须处理	必须 <code>try-catch</code> 或声明 <code>throws</code> , 否则编译不通过	不强制, 可处理可不处理
例子	<code>IOException</code> 、 <code>FileNotFoundException</code> 、 <code>SQLException</code>	<code>NullPointerException</code> 、 <code>ArrayIndexOutOfBoundsException</code> 、 <code>NumberFormatException</code>
引起原因	外部因素: 文件不存在、网络断开、数据库失败	程序 bug: 空指针、越界、类型转换错误

- 简单来说: checked 异常是"你无法控制的外部环境问题", 编译器逼你提前准备; unchecked 异常是"你代码写错了", 编译器不逼你, 但你应该自己修好的bug

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

// 自定义异常: 文件为空
class EmptyFileException extends Exception {
    public EmptyFileException(String message) {
        super(message);
    }
}

public class FileAverage {

    // 读取文件并计算平均值, 可能抛出自定义异常或IO异常
    public static double calculateAverage(String filename)
        throws EmptyFileException, IOException, NumberFormatException {

        // 用 try-with-resources 自动关闭文件资源
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            int sum = 0;        // 总和
            int count = 0;      // 数字个数
        }
    }
}
```

```

String line;
while ((line = reader.readLine()) != null) {
    // 跳过空白行
    if (line.trim().isEmpty()) {
        continue;
    }
    // 尝试解析为整数，解析失败抛出 NumberFormatException
    int num = Integer.parseInt(line.trim());
    sum += num;
    count++;
}

// 如果没读到任何有效数字，抛出“文件为空”异常
if (count == 0) {
    throw new EmptyFileException("文件为空或没有有效的整数内容");
}

// 返回平均值（注意转成 double 避免整数除法）
return (double) sum / count;
}

// try-with-resources 会在结束时自动调用 reader.close()
}

```

```

public static void main(String[] args) {
    String filename = "data.txt";
    try {
        double avg = calculateAverage(filename);
        System.out.println("平均值 = " + avg);
    } catch (EmptyFileException e) {
        System.err.println("错误: " + e.getMessage());
    } catch (NumberFormatException e) {
        System.err.println("错误: 文件中包含无法解析为整数的内容。");
    } catch (IOException e) {
        System.err.println("错误: 无法读取文件 '" + filename + "', 请确认文件是否存在");
    }
}
}

```

Task 4

- `limit = java.util.stream.SliceOps$1@6d06d69c` 这串东西其实是 Stream 对象本身（具体是 `limit` 返回的内部实现类的 `toString()` 结果：类名 + @ + 十六进制哈希码）
- 因为 Stream 是惰性的，它本身不是一个存储了具体数据的容器，而是一个操作流水线的方法 `limit(4)` 只是一个中间的操作，它返回的是一个新的 Stream 对象，记录"我要取前 4 个"这个规则，但此刻并不真正执行、也不真正读取元素 只有调用终结操作如 `collect`、`count` 时，整条流水线才会运行，从源头开始真正遍历元素、逐个处理 类比：Stream 像一条没通电的流水线，你加了 `limit` 只是规划了要装一个限流装置，但机器还没开动。只有当"终结操作"按下开关，流水线才真正运转、产出成品 所以 `System.out.println(limit)` 打印的是"流水线装置本身"，而不是"流水线产出的元素"。

```
import java.util.Arrays;    //导入需要的包
import java.util.List;
import java.util.stream.Collectors;
public class Main {
    public static void main(String[] args) {
        List<Student> students = Arrays.asList(
            new Student("Alice", 85),
            new Student("Bob", 58),
            new Student("Charlie", 90),
            new Student("David", 45),
            new Student("Eve", 72),
            new Student("Frank", 60),
            new Student("Grace", 55),
            new Student("Heidi", 95)
        );

        List<String> passingStudents = students.stream()           // 1. 获取流
            .filter(s -> s.getScore() >= 60)                     // 2. 过滤分数 ≥ 60
            .map(s -> s.getName().toUpperCase())                 // 3. 名字转大写
            .sorted()                                              // 4. 按字母顺序排序
            .collect(Collectors.toList());                        // 5. 收集成 List<String>

        System.out.println(passingStudents);
    }
}
```

结果为[ALICE, CHARLIE, EVE, FRANK, HEIDI]

